Kevin Küster

Zusammenfassung: N-Adress, ALU

N-Adress, ALU

n-Adressmaschinen

Beispiel: $z = x - y$

3-Adress: OP-Code | Ziel | Op1 | Op2

sub z x y | z = x - y

2-Adress: OP-Code | Ziel/Op1 | Op2

sub x y | x = x - y

mov z x | z = x

1,5-Adress: OP-Code | Ziel/Op1 | Op2

=> In jeder Operation liegt min. 1 Operand in einem Register vor

mov h1 x | h1 (Hilfsreg. 1) = x

sub h1 y | h1 = h1 - y

mov z h1 | z = h1

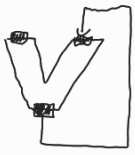
1-Adress: OP-Code | Op1

=> Berechnung via Akku (ALU)

Load X | Akku = X

sub Y | Akku = Akku - Y

stor z | z = Akku



0-Adress: Stackoperation

=> Stack: $\begin{bmatrix} y \\ x \end{bmatrix}$

push X | X

push Y | XY

sub | XY- => X-Y

pop z | z = X - Y

0-Adr.-M.: Geschw.: sehr hoch / Befehlsanzahl: sehr viele => Benötigt mehr speicher, Fehleranfälliger

1-Adr.-M.: Geschw.: hoch / Befehlsanzahl: viele

1,5-Adr.-M.: Geschw.: hoch / Befehlsanzahl: viele

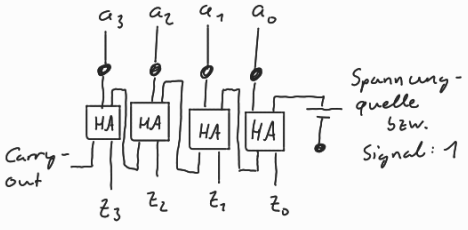
2-Adr.-M.: Geschw.: mittel / Befehlsanzahl: wenige

3-Adr.-M.: Geschw.: gering / Befehlsanzahl: sehr wenige => Weniger Speicher, Einfache(re) Implementierung

ALU

Halbaddierer

Zweierkomplement 4-Bit-Zahl:



Carry ≙ Übertrag



x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Aufbau + FLags

Flags => Steuerung des Programms

Negative-Flag: Vorzeichen -> 1 ≙ -, 0 ≙ +

Zero-Flag: 1 => Alle Bits 0

Carry-Flag: Bei Berechnung ist Übertrag entstanden
↳ MSB (Most significant Bit)

Overflow-Flag: Überschreiten des Zahlenbereichs
=> Überträge bei Bit 8 & 9 versch. (XOR)

